

Proyecto Final

Asistencia a personas en situación de dependencia

ROBOTS MÓVILES

Iker Alaminos Hernández
Nicolás Fernández Blánquez
Klaudia Martínez Bautista
19-1-2025

ÍNDICE

1.	INTRODUCCIÓN	2
2.	PLANTEAMIENTO DEL PROBLEMA	2
3.	ALGORITMOS Y TÉCNICAS IMPLEMENTADAS	3
3.	PROBLEMAS ENCONTRADOS Y SUS SOLUCIONES.....	8
4.	DESCARGA Y PRUEBA DEL PROYECTO	9
5.	PRUEBAS DEL FUNCIONAMIENTO DE LA IMPLEMENTACIÓN.....	10
6.	CONCLUSIÓN	12

1. INTRODUCCIÓN

Al final de la asignatura de robots móviles se plantea un proyecto final, cuyo objetivo es, mediante los conocimientos adquiridos durante el curso académico, implementar un programa relacionado con la asignatura.

Se pueden utilizar técnicas enseñadas durante la asignatura, como la navegación autónoma, la detección de color o el mapeado, así como nuevos algoritmos investigados de forma autodidacta, como la creación de mapas, comandos de voz o detección de caras. Dando acceso también a los robots del laboratorio utilizados durante el curso para realizar algunas prácticas, en caso de ser solicitados.

Así pues, este proyecto simula un problema real y trata de darle una solución lógica y lo más realista posible. Se tratarán puntos como su planteamiento, técnicas y algoritmos utilizados, y problemas que han ido surgiendo con sus respectivas soluciones o posibles soluciones en caso de no haber podido implementarlas.

2. PLANTEAMIENTO DEL PROBLEMA

El presente proyecto tiene como objetivo el desarrollo de un robot de asistencia diseñado para personas en situación de dependencia, como personas de edad avanzada, con algún tipo de enfermedad o discapacidad, dentro de un entorno doméstico. Este robot tiene la capacidad de interactuar con la persona dependiente y otros miembros de la familia mediante un sistema de voz, así como de moverse de manera predefinida en un entorno predeterminado con el uso de tecnología de navegación fija.

A través del uso de un mapa preconfigurado, diseñado y mapeado en Gazebo, el robot se localiza dentro del hogar y puede identificar a los miembros de la familia (en este caso se ha integrado a los miembros del grupo que ha realizado el proyecto, así como al profesor), quienes están representados por colores asignados de forma individual (rojo = Iker, amarillo = Otto, azul = Klau y verde = Nico). El mapa fue creado inicialmente en un entorno simulado (Gazebo), lo que permite verificar el comportamiento del robot en un entorno virtual antes de trasladarlo al mundo real.

Una vez que el robot recibe el comando de la persona dependiente, se desplaza por el hogar siguiendo una ruta por todas las habitaciones, buscando a la persona solicitada y transmitiendo su mensaje a través de un comando de voz. Tras cumplir la tarea, el robot retorna automáticamente a su posición inicial junto a la persona en situación de dependencia.

Este sistema está diseñado para facilitar la vida diaria de las personas dependientes al promover la autonomía en la comunicación con los miembros de su familia en caso de necesitar algún servicio por parte de esta.

3. ALGORITMOS Y TÉCNICAS IMPLEMENTADAS

Para la óptima realización del proyecto se han implementado una serie de técnicas y algoritmos. Estas tecnologías incluyen algunas aprendidas durante la asignatura: la navegación fija, detección de color y retorno a la base, y otras investigadas de manera autodidacta: transmisión de mensajes mediante comandos de voz y creación de mapas. A continuación, se detallan las técnicas y algoritmos, así como su funcionamiento:

Creación del paquete

Con el objetivo de tener todos los códigos y ficheros ordenados en un mismo lugar, lo primero que se hizo fue crear un paquete independiente a los utilizados anteriormente.

```
catkin_create_pkg <nombre_paquete> [dependencias]
```

Una vez se tuvo dicho paquete y se configuró con las dependencias necesarias, se organizó en distintas carpetas como se puede ver a continuación.

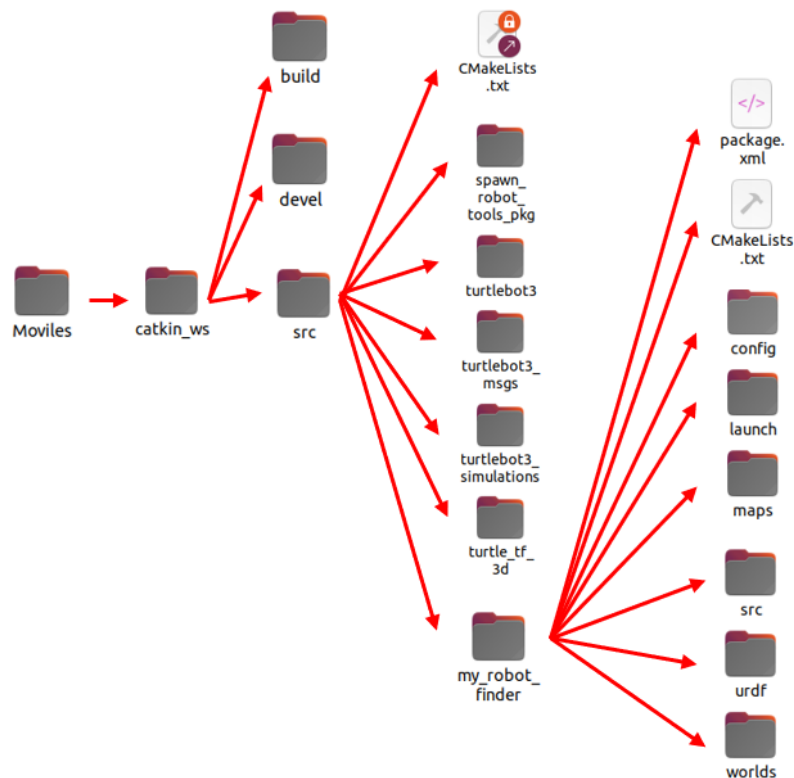


Imagen 1. Distribución del paquete

Se puede ver en la imagen cómo se han aprovechado paquetes propios del turtlebot3 como `spawn_robot_tool_pkg`, `turtlebot3`, `turtlebot3_msgs`, `turtlebot3_simulations` y `turtle_tf_3d`. Además, se ha creado uno nuevo llamado “`my_robot_finder`” que ha sido con el que se ha ido trabajando.

En dicho paquete, se han creado varias carpetas para clasificar los ficheros en función de la tarea que desarrollasen. En la carpeta **config** se han añadido los ficheros encargados de configurar procesos como el algoritmo de localización adaptativo de Montecarlo y el fichero .rviz que crea el espacio de trabajo para que al abrir la aplicación RViz ya esté todo configurado.

En la carpeta **launch** se han clasificado los ficheros .launch encargados de ejecutar los procesos, el fichero más destacado es *mapa.launch* que es el que abre la aplicación de Gazebo con el robot y el mapa configurados.

En cuanto a la carpeta **maps**, esta ha servido para almacenar los ficheros .pgm y .yaml del mapa una vez escaneado. Por otro lado, la carpeta **src** se ha utilizado para clasificar los ficheros .py con el código necesario para que el proyecto cumpliera su función. En esta carpeta se pueden ver los ficheros específicos para la detección de cada uno de los colores como el código principal (*main.py*) donde se declara la máquina de estados con todos los procesos.

Por último, la carpeta **urdf** almacena un fichero .urdf perteneciente a la estructura del turtlebot_burguer que ha sido el que se ha utilizado para la elaboración del proyecto. Por otro lado, en la carpeta **worlds** se ha guardado en un fichero .world el mapa generado en Gazebo.

Creación del mapa

Lo primero que se realizó después de plantear el problema fue crear el mapa con el cual se iba a simular el entorno.

Primeramente, se diseñó y simuló un mapa del entorno doméstico utilizando Gazebo, un simulador robusto que permite crear modelos 3D precisos y probar el comportamiento del robot en un entorno virtual. Se adquirieron diversos diseños predefinidos desde la página oficial de gazebo, como habitaciones enteras o varios conjuntos de muebles. El primer mapa que se creó es el mostrado en la imagen 2.



Imagen 2. Mapa inicial 3D Gazebo

A raíz de una serie de problemas que se explicarán en apartados posteriores, se cambiaron estos diseños predefinidos por bloques sencillos que el propio simulador proporciona, cubos y rectángulos 3D. El mapa final se aprecia en la imagen 3.

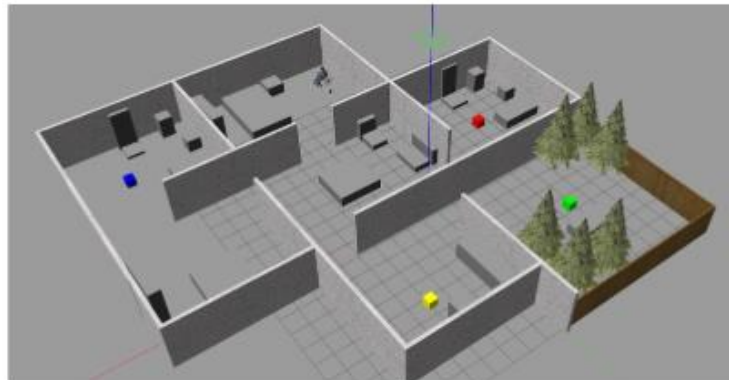
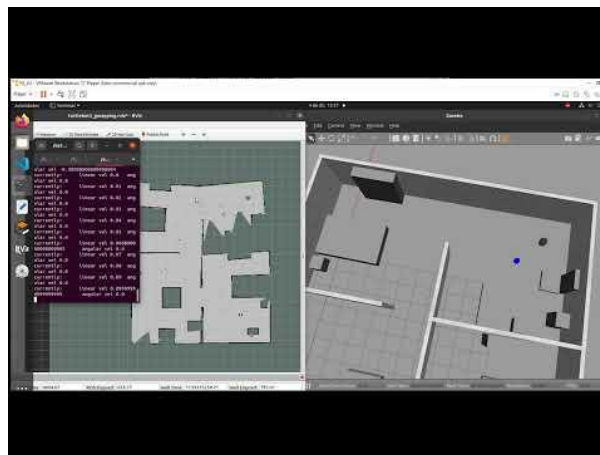


Imagen 3. Mapa utilizado en la simulación

Mapeado del mapa

Una vez creado el mapa del entorno, se procede a mapearlo, es decir, que el robot navegue (controlado por el usuario) por todo el mapa 3D detectando paredes y objetos para crear un mapa en 2D. Se puede observar un fragmento del mapeado en el siguiente vídeo.



Video 1. Proceso de mapeado del entorno

Este mapa será el que posteriormente el robot utilizará para navegar de forma precisa y segura por el entorno simulado, es por esto por lo que es un paso de mucha importancia y que debe realizarse de forma rigurosa para el perfecto funcionamiento del robot de ubicarse y moverse por la casa.

A continuación, se muestra el mapa 2D creado por el robot al terminar de mapear la casa al completo, imagen 5.



Imagen 4. Entorno mapeado

Configuración de la estructura del robot

Como ya se ha comentado, se ha utilizado la estructura del Turtlebot_burguer como base y se ha modificado su fichero .urdf, que define el modelo del robot, para incluir una cámara RGB y un sensor de proximidad. Esto ha permitido al robot contar con la capacidad de detección de distintos colores e interactuar con el entorno.

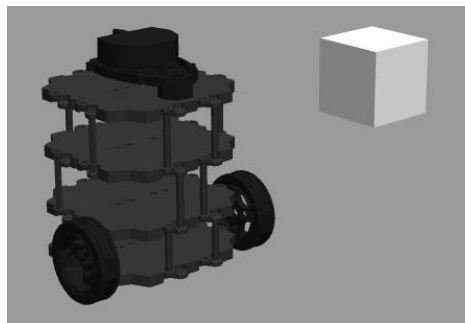


Imagen 5. Cámara RGB instalada al turtlebot3

Navegación

Una vez el robot ha mapeado la casa y ya tiene un mapa de punto de referencia se comenzó con la navegación de este. Se han utilizados las técnicas y los comandos utilizados en las prácticas, en este caso se ha creado un recorrido mediante coordenadas esparcidas por diferentes salas de la casa, siguiendo un camino lógico que podría realizar el robot (este se puede observar en los vídeos de demostración, vídeo 2, vídeo 3, vídeo 4 y vídeo 5).

A continuación, el robot se mueve hacia estas coordenadas mediante la función “*move_base*” utilizada en clase que permite al robot moverse hasta un punto en concreto creando la mejor ruta posible. Además, para evitar que el robot choque o bien con paredes o bien con objetos, se utiliza un sensor de proximidad, utilizamos la información que este nos da para detectar paredes y objetos, y poder evitarlos a una distancia considerable.

Por último, al llegar a cada una de las coordenadas de la ruta, el robot realiza un barrido, es decir, da una vuelta de 360° sobre sí mismo para buscar su objetivo en la sala en la que se encuentre. Así es como el robot se mueve por el entorno en busca de la persona solicitada de manera segura y revisando todas las salas.

Detección de color

Como se ha explicado anteriormente, cada persona se ha asociado a un color, por lo que el robot lo que busca en la simulación son bloques simples de colores distribuidos por el mapa. Para realizar la detección de estos colores se ha utilizado el código de las prácticas, pero esta vez para detectar cuatro colores, el rojo, el amarillo, el azul y el verde. Se ha necesitado ajustar los límites superiores e inferiores de cada uno de los colores para su correcta detección de píxeles.

En cuanto el robot, mediante la cámara implementada y explicada anteriormente, detecta uno de los colores, se para y mediante un comando de voz le dice a la persona que se le necesita y que le siga.

Transmisión de mensajes

Se ha querido implementar esta técnica nueva como algo innovador y con el objetivo de hacer al robot más “humano”. Así, el robot es capaz de comunicar mediante comandos de voz lo que va realizando, como buscar, encontrar y solicitar acciones.

Primeramente, solicita que se ingrese por pantalla el nombre de la persona de la que se requiere ayuda: “Ingrese la persona a buscar (klau, nico, otto, iker) : ”

Nada más la persona en situación de dependencia solicita la ayuda de alguna persona mediante la terminal, este le dice “Buscando a [nombre de la persona solicitada]”.

Una vez el robot ha detectado un color, este espera unos segundos y mediante un comando de voz, utilizando el comando *speak* de la librería gTTS (Google Text-to-Speech), le dice a la persona: “Hola, la persona en situación de dependencia te necesita” “Acompáñame”. Además, al llegar a la base (al lado de la persona en situación de dependencia) dice: “Ya hemos llegado”

Por último, también se ha implementado la detección de voz del propio usuario, es decir, la persona en situación de dependencia solicita la ayuda mediante la voz, diciendo únicamente el nombre de la persona solicitada. Se ha comprobado su correcto funcionamiento, sin embargo, para probar el código y su funcionamiento es más cómodo ingresar estos datos mediante la terminal, ya que la detección no es perfecta y el ruido del ambiente puede interferir.

Retorno a la base

Una vez el robot ha detectado el color y mandado su mensaje mediante el comando de voz, este debe volver al lado de la persona en situación de dependencia. Esto se realiza con la función “*move_base*”, al igual que se ha utilizado para la navegación, esta consiste en mover al robot a una coordenada fija creando la mejor ruta posible, en este caso el punto inicial desde donde comienza la simulación el robot.

3. PROBLEMAS ENCONTRADOS Y SUS SOLUCIONES

Durante la realización del proyecto han ido surgiendo diferentes problemas y dificultades. Algunos de estos se han podido solventar cambiando una serie de cosas, y para otros ha sido necesario reestructurar toda esa sección, incluso, se han debido descartar ideas y cambiarlas por otras.

A continuación, se comentarán tanto los problemas como las soluciones o posibles soluciones encontradas e implementadas durante la realización del proyecto.

Diseño y creación del mapa

El problema que más tiempo ha llevado detectar y solventar está relacionado con el diseño, y sobre todo la creación del mapa. Inicialmente, se utilizaron modelos de habitaciones de la propia aplicación de Gazebo, imagen 2, lo que generó dos inconvenientes significativos: el robot detectaba la caja de colisión de la habitación completa en lugar de los muebles individuales, y el mapa diseñado consumía muchos recursos, provocando problemas de rendimiento en la máquina virtual.

Para solucionar este problema se diseñó un nuevo mapa sustituyendo los muebles por bloques creados manualmente, imagen 3, como cuadrados o rectángulos simples. Esto permite un diseño más eficiente que mejoró tanto el rendimiento computacional como la interacción del robot con el entorno.

Mapeado

El Turtlebot enfrentaba dificultades para realizar el mapeado en áreas grandes donde no existían paredes ni muebles cercanos, puesto que el laser no rebotaba sobre ninguna superficie y se dejaba huecos del mapa sin mapear.

La solución que se implementó fue añadir nuevos muebles/bloques en estas zonas que causaban problemas, como el jardín y la parte central de la casa, para proporcionar puntos de referencia al sistema de mapeado y mejorar su detección.

Funcionamiento del láser

A la hora de navegar, el robot se chocaba con objetos al no detectarlos correctamente, así como paredes, muebles o cualquier otro obstáculo o bloque debido a un fallo en el sensor láser.

Tras analizar el problema y revisar los parámetros de los objetos del entorno, se descubrió que el error estaba en la suscripción a los *topics* del láser. Se corrigió la configuración, asegurándose de suscribirse al *topic* adecuado, lo que permitió que el láser funcionara como se esperaba.

Navegación aleatoria

El problema de la navegación fue difícil de gestionar, puesto que ya estaba casi terminado el proyecto, pero durante la búsqueda de una persona, el robot realizaba movimientos completamente aleatorios. Esto ocasionaba que quedara atrapado en la misma habitación durante largos periodos de tiempo o que volviera repetidamente a habitaciones ya exploradas, incrementando significativamente el tiempo de búsqueda.

Por lo tanto, se tuvo que escoger entre dejarlo como estaba, aunque el tiempo de búsqueda fuese enorme e incluso no llegase a encontrar a nadie, o cambiar radicalmente el sistema de navegación, se optó por esto último.

La decisión que se tomó fue implementar rutas fijas a través de coordenadas del mapa y optimizadas que garantizan que el robot recorra todas las habitaciones de manera ordenada y eficiente, al llegar a cada punto se realiza un barrido, es decir, una vuelta sobre sí mismo de 360°, minimizando tiempos de búsqueda y evitando exploraciones redundantes.

Detección de voz

Se implementó un sistema de detección de voz para que el usuario se comunicase con el robot a través de comandos de voz. Se solicitaba la ayuda mediante la voz, así como el robot también transmitía los mensajes a través de estos. El problema estaba relacionado con la detección de la voz del usuario, puesto que el ruido dificultaba el entendimiento.

Se tomó la decisión de conservar los comandos de voz del robot, pero respecto a la detección de voz del usuario, se dejó que se introdujese por la terminal la orden de búsqueda. No obstante, se ha dejado comentado en el código la opción de la detección de voz por si se quisiera probar, funciona, pero se podría mejorar con un gran filtro de palabras parecidas a los nombres de los familiares.

4. DESCARGA Y PRUEBA DEL PROYECTO

El proyecto se ha subido a la plataforma GitHub, el objetivo de subir un proyecto a GitHub es hacerlo más accesible para otros usuarios. Para poder acceder al contenido de dicho proyecto desde una máquina local, es necesario descargarlo. Este proceso se realiza clonando el repositorio a través de la plataforma GitHub utilizando Git, una herramienta de control de versiones ampliamente utilizada. A continuación, se describe el procedimiento para clonar un proyecto de GitHub a través de la terminal de un sistema Linux.

En primer lugar, se debe dirigir a la carpeta donde se desea descargar el proyecto. Una vez allí, es necesario abrir una terminal y ejecutar el siguiente comando para descargar una copia completa del proyecto en la carpeta actual.

```
git clone https://github.com/niicoofdezz/Proyecto\_Robots\_Moviles/tree/main
```

Con esto, se creará una nueva carpeta con el nombre del repositorio. Para verificar que se ha clonado correctamente, se puede ejecutar *ls* y comprobar que se encuentran todas las carpetas.

Una vez se tiene el proyecto descargado, es necesario conocer cómo hacerlo funcionar. Para ello, se utilizan los siguientes comandos.

```
roslaunch my_robot_finder mapa.launch
```

```
roslaunch my_robot_finder turtlebot3_navigation.launch
```

Estos comandos abrirán tanto Rviz como Gazebo, donde se ejecutará el proyecto. Tras tener los entornos de ejecución abiertos y preparados, es necesario abrir una nueva terminal donde ejecutar el código principal de nuestro programa, es necesario estar en la siguiente ruta:

```
~/Moviles/catkin_ws/src/my_robot_finder/src
```

```
python3 main.py
```

Este código, pedirá al usuario indicar el nombre de una de las cuatro personas a buscar. Una vez introducido, el usuario deberá abrir una nueva terminal y ejecutar el código correspondiente a la persona solicitada, es decir, uno de los cuatro comandos siguientes:

```
python3 klau.py
```

```
python3 iker.py
```

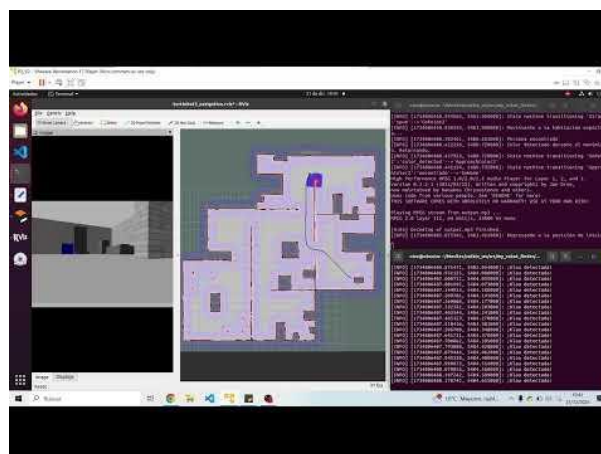
```
python3 nico.py
```

```
python3 otto.py
```

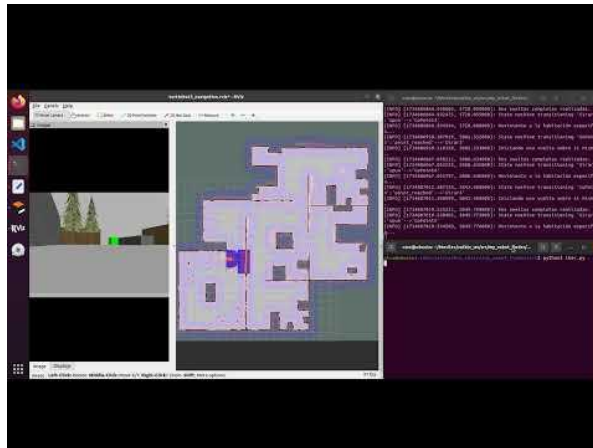
Con esto, el proyecto comenzará a funcionar y realizará su función.

5. PRUEBAS DEL FUNCIONAMIENTO DE LA IMPLEMENTACIÓN

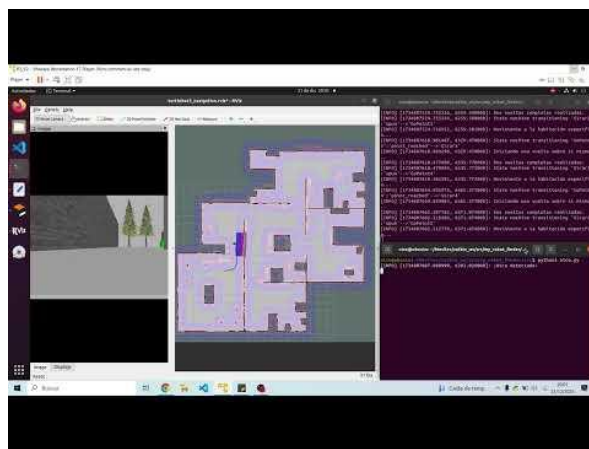
Por último, se han grabado una serie de vídeos para demostrar la correcta funcionalidad del proyecto, en estos se muestran varias funciones de manera satisfactoria.



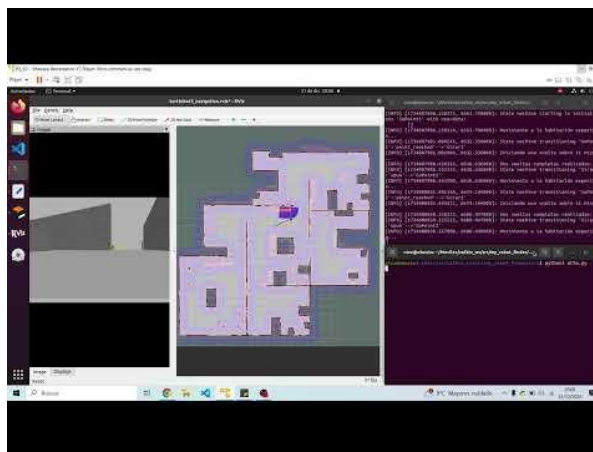
Video 2. Simulación Buscando a Klau



Video 3. Simulación buscando a Iker



Video 4. Simulación buscando a Nico



Video 5. Simulación buscando a Otto

Al comienzo de cada uno de los vídeos se muestra cómo se ejecuta el código maestro, “main.py”, donde se encuentra la máquina de estados, este pide mediante un comando de voz, que se ingrese por pantalla el nombre de la persona a buscar. En el vídeo 2 se busca a “Klau”, la cual es la más rápida de encontrar por la disposición de la ruta establecida, se debe ejecutar, tal y como se ve en el vídeo, simultáneamente el código que detecta los píxeles de color de cada persona, en caso de buscar a Klau, se lanzará el programa “klau.py”, el cual detecta el color azul.

Una vez detectado el color el robot se detiene y dice “Hola, la persona en situación de dependencia te necesita”, seguidamente vuelve a la base, supuestamente la persona ahora le seguiría hasta donde se encuentra la persona necesitada.

En los vídeos 3, 4 y 5, se puede observar una navegación mediante coordenadas más amplia, puesto que los bloques están más alejados. Se observa como el robot sigue la ruta, punto por punto mientras hace barridos en busca de las personas, manda mensajes y regresa junto a la persona en situación de dependencia.

6. CONCLUSIÓN

En conclusión, se ha abordado el proyecto final de la asignatura robots móviles haciendo uso de técnicas aprendidas a lo largo del curso, así como conocimientos investigados de manera autónoma.

Se ha planteado una situación y un problema real, y se le ha aplicado una solución acorde con la temática solicitada. Se han implementado varios programas haciendo uso de un robot turtlebot para que navegue, detecte colores y se comunique mediante comandos de voz para asistir a una persona en situación de dependencia.

Además, se han detectado problemas en la implementación y se han arreglado haciendo uso de otras técnicas o de los conocimientos explicados y utilizados en clase.

En definitiva, se ha llegado al objetivo propuesto de una manera satisfactoria y funcional, puesto que la tarea planteada inicialmente se cumple de una manera eficiente y óptima realizando las funciones pertinentes de una forma correcta y ordenada.